

Atividade Prática: Busca Sequencial e por RRN

Organização e Recuperação de Dados

Prof. Jonathan

Profa. Valéria

UEM – CTC – DIN

Programa 1

- Implemente o programa *busca_id*
- *busca_id* faz uma busca sequencial em um arquivo de registros de tamanho variável com um campo indicador de tamanho de 2 bytes
- A busca será realizada no arquivo **peessoasGOT.dat**
 - Nesse arquivo, cada registro possui os seguintes campos:
 - Tamanho (fixo), identificador, sobrenome, nome, nome do castelo da família ou de residência, cidade, região de Westeros/Essos
 - O campo **identificador** está armazenado como *string* e é a **chave primária**

Exemplo de registros com **indicador de tamanho no início**

6133|Hightower|Alicent|Red Keep|King's Landing|Terras da Coroa|**43**31|Stark|Bran|...

2 bytes
little endian

61 bytes

Programa 1

- Inicialmente, o programa deve solicitar o identificador a ser buscado
- Se o registro correspondente for encontrado, o mesmo deve ser apresentado em tela, finalizando a execução
- Caso contrário, o programa deve imprimir uma mensagem de registro não encontrado e terminar

```
> ▾ TERMINAL
🔌
● vdfeltrim@Notebook2025:/mnt/c/Users/valer/Desktop/ORD/Pratica 2 - Busca sequencial e direta$ py3 busca_seq.py
Digite o ID a ser buscado: 33
Campo #1: 33
Campo #2: Hightower
Campo #3: Alicent
Campo #4: Red Keep
Campo #5: King's Landing
Campo #6: Terras da Coroa

● vdfeltrim@Notebook2025:/mnt/c/Users/valer/Desktop/ORD/Pratica 2 - Busca sequencial e direta$ py3 busca_seq.py
Digite o ID a ser buscado: 62
0 registro de ID 62 não foi encontrado

○ vdfeltrim@Notebook2025:/mnt/c/Users/valer/Desktop/ORD/Pratica 2 - Busca sequencial e direta$
```

Busca sequencial
em um arquivo com
registros de tamanho
variável

A função **leia_reg()**
mencionada neste
pseudocódigo é a
mesma que foi utilizada
no programa
le_registros.py

Pseudo busca_id

PROGRAMA: **busca_id**

abra o arquivo 'pessoasGOT.dat' para leitura binária e chame-o
de ENTRADA

leia o identificador a ser buscado em CHAVE

inicialize ACHOU com *falso*

chame a função leia_reg(ENTRADA) e armazene o retorno em REG

enquanto REG for diferente de vazio **e** não ACHOU **faça**

faça ID receber o 1º campo de REG

id = reg.split(sep='|')[0]

se ID é igual a CHAVE **então**

ACHOU recebe *verdadeiro*

senão chame leia_reg(ENTRADA) e armazene em REG

se ACHOU **então**

Lembre-se que reg.split(sep='|') retorna

uma lista com os campos contidos em REG

para cada CAMPO em REG **faça**

escreva CAMPO na tela

senão imprima uma mensagem de erro

feche ENTRADA

fim PROGRAMA

Programa 2

- Implemente o programa *busca_sobrenome*
- *busca_sobrenome* também faz uma busca sequencial no arquivo **peessoasGOT.dat**, porém utiliza o campo **Sobrenome** como chave
 - Sobrenome é uma **chave secundária**, então a lógica da busca deve ser outra: todos os registros devem ser lidos sempre, para garantir que todas as ocorrências da chave buscada foram localizadas

```
> ▾ TERMINAL
vdfeltrim@Notebook2025:/mnt/c/Users/valer/Desktop/ORD/Pratica 2 - Busca sequencial e direta$ py3 busca_sobrenome.py
Digite o sobrenome a ser buscado: Tully
Campo #1: 21
Campo #2: Tully
Campo #3: Edmure
Campo #4: Riverrun
Campo #5: Riverrun
Campo #6: Terras Fluviais

Campo #1: 20
Campo #2: Tully
Campo #3: Catelyn
Campo #4: Riverrun
Campo #5: Riverrun
Campo #6: Terras Fluviais
```

Programa 3

- Implemente o pseudocódigo *busca_rrn*
- O programa *busca_RRN* faz uma busca direta no arquivo **peessoasGOTfixo.dat**
 - Esse arquivo usa registros de tamanho fixo com 64 bytes
 - Registros de tamanho fixo com campos de tamanho variável
 - As sobras de espaço dentro de um registro são ser preenchidas com o caractere '\0'
 - O arquivo inicia com um **cabeçalho de 4 bytes little endian** que armazena o **total de registros** do arquivo

[illegible]

Programa 3

- No arquivo **peessoasGOTfixo.dat** os registros estão ordenados pelo ID
 - Assim, o registro de ID = 1 está no RRN = 0; o de ID = 2 está no RRN = 1; e assim por diante

```
● vdfeltrim@Notebook2025:/mnt/c/Users/valer/Desktop/ORD/Pratica 2 - Busca sequencial e direta$ py3 busca_rrn.py
Digite o RRN a ser buscado: 5
Campo #1: 6
Campo #2: Lannister
Campo #3: Jaime
Campo #4: Casterly Rock
Campo #5: Lannisport
Campo #6: Terras Ocidentais
```

Busca direta
em um arquivo de
registros de tamanho
fixo no formato
explicado no slide
anterior

O número **total de registros** no arquivo
está gravado no
cabeçalho como um
número de 4 bytes
com byteorder = 'little'

PROGRAMA: **busca_rrn**

abra 'pessoasG0Tfixo.dat' para leitura binária e chame-o de
ENTRADA

leia 4 bytes de ENTRADA e armazene em CAB

Converta CAB para inteiro armazene em TOTAL_REG

utilize o método int.from_bytes() para converter CAB

leia o RRN do registro a ser recuperado

se RRN >= TOTAL_REG **então**

imprima uma msg de erro e termine

calcule o OFFSET de leitura */* RRN * 64 + 4 */*

faça seek para OFFSET

leia o registro e decodifique-o em REG

utilize o método decode() para decodificar REG

Lembre-se que reg.split(sep='|') retorna

uma lista com os campos contidos em REG

para cada CAMPO em REG **faça**

escreva CAMPO na tela

feche ENTRADA

fim PROGRAMA

Programa 4

- Implemente o programa *busca_e_atualiza*
- Diferentemente dos programas feitos até agora, o programa *busca_e_atualiza* permite:
 - Abertura de um arquivo existente (ou se crie, caso não exista)
 - **Inserção** de novos registros (sempre no final do arquivo)
 - **Busca um registro pelo seu RRN para alterações** (busca e possível regravação do registro)
- Para permitir a busca por RRN, o programa deve utilizar **registros de tamanho fixo**
 - Adotaremos o mesmo formato de **registros de tamanho fixo com campos de tamanho variável utilizado pelo Programa 3**

Programa 4

Digite o nome do arquivo a ser aberto: pessoasGOTfixo.dat

PROGRAMA PARA INSERCAO E ALTERACAO DE REGISTROS

Suas opcoes sao:

1. Inserir um novo registro
2. Buscar um registro por RRN para alteracoes
3. Terminar o programa

Digite o numero da sua opcao: █

Programa 4

PROGRAMA PARA INSERCAO E ALTERACAO DE REGISTROS

Suas opcoes sao:

1. Inserir um novo registro
2. Buscar um registro por RRN para alteracoes
3. Terminar o programa

Digite o numero da sua opcao: 1

Digite os dados do novo registro:

ID: 36

Sobrenome: Mormont

Nome: Lyanna

Castelo: Mormont Keep

Cidade: Bear Island

Região: Norte

Programa 4

PROGRAMA PARA INSERCAO E ALTERACAO DE REGISTROS

Suas opcoes sao:

1. Inserir um novo registro
2. Buscar um registro por RRN para alteracoes
3. Terminar o programa

Digite o numero da sua opcao: 2

Digite o RRN do registro: 6

Conteudo do registro

7
Targaryen
Daenerys
Great Pyramid
Meereen
Baía dos Escravos

Voce quer modificar este registro?

Responda S ou N, seguido de <Enter> ==> █

Programa 4

PROGRAMA PARA INSERCAO E ALTERACAO DE REGISTROS

Suas opcoes sao:

1. Inserir um novo registro
2. Buscar um registro por RRN para alteracoes
3. Terminar o programa

Digite o numero da sua opcao: 2

Digite o RRN do registro: 6

Conteudo do registro

7
Targaryen
Daenerys
Great Pyramid
Meereen
Baía dos Escravos

Voce quer modificar este registro?

Responda S ou N, seguido de <Enter> ==> S
Digite os dados do novo registro:

ID: 7
Sobrenome: Targaryen
Nome: Daenerys
Castelo:

Abertura ou criação
um arquivo de
registros de
tamanho fixo (64b)

Os registros podem
ser inseridos
ou buscados

Registros são
buscados
por RRN

Armazene o número
total de registros no
cabeçalho do arquivo
usando 4 bytes

O cabeçalho deve
ser gravado na
criação do arquivo e
atualizado ao final de
cada execução

PROGRAMA: **busca_e_atualiza**

receba o nome do arquivo em NOME_ARQ

abra o arquivo NOME_ARQ para L/E binária e chame-o de ARQ # *'r+b'*

se o arquivo NOME_ARQ não existir **então**

crie e abra NOME_ARQ para L/E binária com o nome lógico ARQ # *'w+b'*

faça TOTAL_REG = 0 e escreva-o em ARQ com 4 bytes

senão leia o cabeçalho de 4 bytes de ARQ e armazene-o em TOTAL_REG

receba a escolha do usuário em OPCA0 #*(1)inserir (2)buscar (3)sair*

enquanto (OPCA0 < 3) **faça**

se OPÇÃO == 1:

receba todos os campos do registro e concatene em REG

codifique e preencha REG até 64b

use encode() para codificar e ljust(64, b'\0') para preencher

calcule o OFFSET de gravação # *TOTAL_REG * 64 + 4*

faça seek para OFFSET e escreva REG em ARQ

incremente TOTAL_REG

senão se OPCA0 == 2:

receba o RRN

se RRN >= TOTAL_REG imprima msg de erro

senão calcule o OFFSET de leitura # *RRN * 64 + 4*

faça seek para OFFSET

leia o registro REG e mostre-o na tela

receba a opção sobre uma possível alteração em ALTERAR

se ALTERAR **então**

receba todos os campos do registro e concatene em REG

codifique e preencha REG até 64b

faça seek para OFFSET e escreva REG em ARQ

receba a escolha do usuário em OPCA0

fim enquanto -- CONTINUAÇÃO NO PRÓXIMO SLIDE

Abertura ou criação
um arquivo de
registros de
tamanho fixo (64b)

Os registros podem
ser inseridos
ou buscados

Registros são
buscados
por RRN

Armazene o número
total de registros no
cabeçalho do arquivo
usando 4 bytes

O cabeçalho deve
ser gravado na
criação do arquivo e
atualizado ao final de
cada execução

```
# -- CONTINUAÇÃO busca_e_atualiza  
faça seek para o início de ARQ  
escreva TOTAL_REG em ARQ com 4 bytes  
feche ARQ  
fim PROGRAMA
```